

GLOSARIUM SISTEM

Cloud-Based Data Processing & Monitoring Platform

Kelompok 11 — Tugas Akhir Cloud Computing 2026

Dokumen ini disusun sebagai panduan bagi pembaca awam yang ingin memahami istilah-istilah teknis yang digunakan dalam proyek ini. Setiap istilah dijelaskan dengan bahasa yang sederhana dan mudah dipahami, tanpa perlu memiliki latar belakang teknis terlebih dahulu. Pada bagian akhir dokumen, disajikan penjelasan menyeluruh tentang alur kerja sistem secara lengkap.

BAGIAN I — KAMUS ISTILAH

A — Infrastruktur & Layanan Cloud

Action Group (Azure Monitor)	Sekelompok tindakan yang akan dijalankan secara otomatis ketika sebuah peringatan (alert) terpicu. Contoh tindakan yang dikonfigurasi pada proyek ini adalah pengiriman notifikasi surel kepada administrator sistem.
Alert Rule (Azure Monitor)	Aturan yang mendefinisikan kondisi ambang batas (threshold) pada sebuah metrik tertentu. Jika metrik melewati ambang yang telah ditetapkan, sistem secara otomatis memicu Action Group. Contoh pada proyek ini: peringatan dibunyikan jika jumlah HTTP Error 5xx melebihi 5 kejadian dalam 5 menit.
App Service Plan	Paket infrastruktur di Microsoft Azure yang menentukan kapasitas, sistem operasi, dan harga untuk menjalankan sebuah Web App atau Function App. Proyek ini menggunakan dua plan: Consumption Plan (bayar per eksekusi) untuk Azure Functions, dan F1 Free Tier untuk App Service cadangan.
Application Insights	Layanan pemantauan performa aplikasi (APM) dari Microsoft Azure yang secara otomatis mencatat setiap permintaan masuk, durasi respons, kegagalan, jejak pengecualian (exception trace), dan metrik CPU/memori dari backend. Ibaratnya, ini adalah 'kotak hitam' atau CCTV untuk aplikasi yang berjalan di cloud.

Azure Blob Storage	Layanan penyimpanan objek/berkas dari Microsoft Azure yang cocok untuk menyimpan data tidak terstruktur dalam jumlah besar, seperti berkas CSV, JSON, gambar, dan video. Dalam proyek ini digunakan untuk menyimpan salinan berkas mentah yang diunggah pengguna ke dalam kontainer bernama 'raw-data'.
Azure Cosmos DB	Layanan basis data NoSQL (non-relasional) yang dikelola penuh oleh Microsoft Azure. Cosmos DB menyimpan data dalam format dokumen JSON yang fleksibel dan mendukung distribusi global. Dalam proyek ini digunakan untuk menyimpan data telemetri hasil pemrosesan dan data akun pengguna (login/registrasi).
Azure Functions	Layanan komputasi tanpa server (serverless) dari Microsoft Azure yang menjalankan kode hanya saat ada permintaan masuk atau peristiwa yang memicunya. Tidak ada server yang perlu dikelola; sistem akan 'bangun' saat dibutuhkan dan 'tidur' kembali setelahnya. Ini adalah otak pemrosesan utama pada proyek ini, dibangun dengan Python 3.11.
Azure Key Vault	Layanan penyimpanan rahasia (secret), kunci enkripsi, dan sertifikat digital yang dikelola secara terpusat dan aman oleh Microsoft Azure. Dalam proyek ini, Key Vault menyimpan string koneksi Cosmos DB dan kunci rahasia untuk penandatanganan token JWT, sehingga informasi sensitif tersebut tidak perlu ditulis secara langsung di dalam kode sumber.
Azure Monitor	Platform pemantauan terpusat dari Microsoft Azure yang mengumpulkan metrik (angka-angka performa) dan log (catatan kejadian) dari seluruh layanan Azure yang digunakan. Bekerja sama dengan Application Insights dan Log Analytics untuk memberikan gambaran menyeluruh tentang kesehatan sistem.
Azure Traffic Manager	Layanan penyeimbang beban (load balancer) berbasis DNS dari Microsoft Azure yang bekerja pada lapisan jaringan global. Dalam proyek ini dikonfigurasi dengan metode 'Priority Routing': seluruh lalu lintas diarahkan ke Azure Functions (prioritas 1), dan secara otomatis beralih ke App Service cadangan (prioritas 2) apabila backend utama terdeteksi tidak responsif.
Azure App Service	Layanan platform-as-a-service (PaaS) dari Microsoft Azure untuk menjalankan aplikasi web berbasis HTTP. Dalam proyek ini digunakan sebagai backend cadangan (secondary backend) yang menjalankan aplikasi Node.js minimalis untuk memberikan respons saat backend utama (Azure Functions) sedang tidak tersedia.

B — Pemrosesan & Kualitas Data

Blob Trigger	Mekanisme pemicu (trigger) otomatis pada Azure Functions yang aktif setiap kali sebuah berkas baru ditulis ke dalam kontainer Azure Blob Storage. Artinya, pengguna tidak perlu memanggil API secara manual; cukup letakkan berkas di storage, dan backend akan memproses berkas tersebut secara otomatis.
---------------------	--

Boolean Conversion	Proses konversi nilai teks seperti 'yes', 'no', 'true', 'false', '1', '0' menjadi tipe data boolean (True/False) yang sesungguhnya. Proses ini dilakukan secara otomatis oleh mesin pembersih data pada platform agar tipe data di basis data konsisten dan dapat diolah secara matematis.
Cleaning Report	Laporan detail yang dikembalikan oleh sistem setelah proses pembersihan data selesai. Laporan ini memuat informasi jumlah rekaman sebelum dan sesudah pembersihan, jumlah nilai yang dikonversi, jumlah duplikat yang dihapus, dan jumlah pencilan (outlier) yang disaring.
Data Profiling	Proses pemeriksaan awal terhadap sebuah kumpulan data untuk menghasilkan ringkasan statistik, seperti jumlah baris, jumlah kolom, tipe data setiap kolom, persentase nilai kosong, dan jumlah duplikat. Fitur ini tersedia melalui endpoint /api/analyze pada platform.
Duplikat (Duplicate Rows)	Baris-baris data yang memiliki isi yang identik pada semua kolomnya. Data duplikat dianggap tidak memberikan informasi tambahan dan dapat mendistorsi hasil analisis statistik, sehingga dihapus secara otomatis saat pengguna mengaktifkan mode pembersihan data.
IQR (Interquartile Range)	Ukuran statistik yang merepresentasikan rentang antara kuartil pertama (Q1) dan kuartil ketiga (Q3) dari sebuah distribusi data numerik. Rumus: $IQR = Q3 - Q1$. Data yang nilainya berada di bawah ($Q1 - 1.5 \times IQR$) atau di atas ($Q3 + 1.5 \times IQR$) dianggap sebagai pencilan (outlier) dan disaring oleh sistem.
Missing Value (Nilai Kosong)	Sel atau kolom dalam sebuah dataset yang tidak memiliki nilai sama sekali. Nilai kosong dapat menyebabkan error saat data diolah secara matematis. Sistem secara otomatis mengonversi sel kosong menjadi 'null' (nilai kosong yang terstandarisasi) agar dapat ditangani dengan konsisten oleh basis data.
Numeric Conversion	Proses konversi nilai teks yang merepresentasikan angka (misalnya '3.14' atau '100') menjadi tipe data numerik (float atau integer) yang sesungguhnya. Konversi ini penting agar operasi matematika seperti rata-rata, jumlah, dan perbandingan dapat dilakukan dengan benar.
Outlier (Pencilan)	Nilai data yang menyimpang sangat jauh dari keseluruhan distribusi data lainnya. Outlier dapat disebabkan oleh kesalahan pengukuran, kesalahan entri data, atau memang merupakan kejadian yang sangat tidak biasa. Sistem menggunakan metode IQR untuk mendeteksi dan menyaring outlier secara otomatis.
Telemetry	Data pengukuran yang dikumpulkan dari perangkat, sensor, atau sistem secara otomatis dan dikirimkan ke server untuk dianalisis. Dalam konteks proyek ini, telemetry merujuk pada data dari perangkat IoT atau log sistem yang diunggah dalam format CSV/JSON untuk diproses dan disimpan.

Trimming (Pemangkasan Spasi)	Proses penghapusan karakter spasi, tab, atau baris baru yang tidak perlu di awal dan akhir sebuah nilai teks. Misalnya, ' sensor_01 ' akan dibersihkan menjadi 'sensor_01'. Langkah ini penting untuk menghindari inkonsistensi saat melakukan perbandingan atau pencarian data.
-------------------------------------	--

C — Keamanan & Autentikasi

Authorization Header	Bagian dari permintaan HTTP yang digunakan untuk menyampaikan informasi kredensial kepada server. Dalam proyek ini, setiap permintaan ke endpoint yang diproteksi harus menyertakan header 'Authorization: Bearer ' agar server dapat mengidentifikasi dan memverifikasi identitas pengguna.
Bearer Token	Skema autentikasi HTTP di mana akses ke sumber daya diberikan kepada siapa saja yang 'membawa' (bear) token yang valid. Token ini biasanya berupa JWT dan disertakan pada setiap permintaan API di dalam Authorization header.
Defense in Depth (Pertahanan Berlapis)	Strategi keamanan yang menerapkan beberapa lapisan mekanisme perlindungan secara bersamaan. Gagasannya adalah bahwa jika satu lapisan berhasil ditembus oleh penyerang, lapisan-lapisan berikutnya masih akan memberikan perlindungan. Proyek ini menerapkannya melalui Cloudflare Proxy, RBAC, Key Vault, dan VNet NSG.
Function Key	Kunci API (API Key) unik yang diterbitkan oleh Azure Functions untuk mengautentikasi permintaan masuk ke tingkat endpoint tertentu. Dalam proyek ini, Function Key tidak pernah diekspos langsung ke browser pengguna; kunci tersebut disimpan sebagai variabel lingkungan rahasia di sisi Cloudflare dan disertakan secara otomatis oleh proxy.
Hash / Password Hash	Hasil dari proses transformasi satu arah (one-way) terhadap sebuah kata sandi menggunakan algoritma kriptografis. Setelah di-hash, kata sandi asli tidak dapat dipulihkan (irreversible). Proyek ini menggunakan algoritma PBKDF2-SHA256 dengan 160.000 iterasi untuk menghasilkan hash yang sangat kuat dan tahan terhadap serangan brute-force.
JWT (JSON Web Token)	Standar terbuka (RFC 7519) untuk membuat token akses yang berisi klaim (informasi) pengguna yang ditandatangani secara digital. Token JWT terdiri dari tiga bagian yang dipisahkan titik: Header (algoritma), Payload (data pengguna seperti ID, peran, dan waktu kedaluwarsa), dan Signature (tanda tangan digital). Bersifat stateless karena server tidak perlu menyimpan sesi.
Managed Identity	Fitur keamanan Azure yang memungkinkan sebuah layanan (seperti Azure Functions) untuk mengautentikasi dirinya ke layanan Azure lain (seperti Key Vault atau Azure Monitor) tanpa perlu menggunakan nama pengguna atau kata sandi. Identitasnya dikelola sepenuhnya oleh platform Azure.

NSG (Network Security Group)	Daftar aturan keamanan jaringan di Microsoft Azure yang berfungsi seperti firewall virtual. NSG mengontrol lalu lintas jaringan masuk (inbound) dan keluar (outbound) dari sumber daya Azure berdasarkan protokol, port, dan alamat IP sumber/tujuan.
PBKDF2-SHA256	Singkatan dari Password-Based Key Derivation Function 2 dengan algoritma hash SHA-256. Ini adalah metode hashing kata sandi yang direkomendasikan secara industri karena menggunakan 'salt' (nilai acak unik) dan ribuan iterasi pengulangan untuk mempersulit serangan kamus (dictionary attack) dan brute-force.
RBAC (Role-Based Access Control)	Sistem pengaturan hak akses yang membatasi apa yang dapat dilakukan oleh pengguna berdasarkan 'peran' (role) yang dimilikinya. Dalam proyek ini terdapat tiga peran: 'user' (pengguna umum yang hanya bisa mengelola datanya sendiri), 'admin' (dapat mengelola semua pengguna dan data), dan 'dev' (dapat melihat metrik operasional sistem).
Salt (Kriptografi)	Nilai acak unik yang ditambahkan ke kata sandi sebelum proses hashing dilakukan. Salt memastikan bahwa dua pengguna dengan kata sandi yang sama akan menghasilkan hash yang berbeda, sehingga serangan menggunakan tabel hash yang sudah dihitung sebelumnya (rainbow table attack) menjadi tidak efektif.
Zero-Exposure Architecture	Prinsip desain arsitektur di mana komponen-komponen sensitif dari sistem (seperti URL backend, API key, dan connection string) tidak pernah terekspos langsung ke klien (browser). Pada proyek ini diwujudkan dengan menggunakan Cloudflare Pages Function sebagai proxy yang menyembunyikan URL dan kunci Azure Functions dari pengguna akhir.

D — Jaringan & Distribusi Konten

CDN (Content Delivery Network)	Jaringan server yang tersebar di berbagai lokasi geografis di seluruh dunia yang berfungsi untuk menyajikan konten statis (gambar, CSS, JavaScript) dari lokasi yang paling dekat dengan pengguna. Ini mengurangi latensi dan mempercepat waktu muat halaman. Cloudflare menyediakan CDN global untuk frontend proyek ini.
Cloudflare DNS	Layanan Domain Name System (DNS) dari Cloudflare yang menerjemahkan nama domain yang mudah dibaca manusia (seperti kelompok11cc.my.id) menjadi alamat IP yang dimengerti komputer. Selain fungsi DNS dasar, Cloudflare DNS juga menyediakan perlindungan DDoS dan penyembunyian alamat IP server asal.
Cloudflare Pages	Platform hosting bertenaga CDN global dari Cloudflare yang dikhususkan untuk melayani situs web dan aplikasi statis (HTML, CSS, JavaScript). Platform ini terintegrasi dengan GitHub untuk melakukan deployment otomatis setiap kali ada perubahan kode yang diunggah.

Cloudflare Pages Function	Fungsi serverless yang berjalan di sisi edge (jaringan Cloudflare) dan dapat digunakan untuk menambahkan logika sisi server ke dalam proyek Cloudflare Pages. Dalam proyek ini digunakan sebagai proxy yang menangkap semua permintaan ke path <code>/api/*</code> , menambahkan Function Key Azure secara rahasia, dan meneruskannya ke backend Azure.
DNS (Domain Name System)	Sistem yang berfungsi seperti 'buku telepon' internet. DNS menerjemahkan nama domain yang mudah diingat manusia (contoh: google.com) menjadi alamat IP numerik yang dipahami oleh komputer untuk membuat koneksi jaringan.
DNS TTL (Time To Live)	Nilai waktu (dalam detik) yang menentukan berapa lama sebuah resolver DNS menyimpan (men-cache) jawaban dari sebuah permintaan DNS sebelum ia perlu meminta ulang ke server DNS otoritatif. TTL yang rendah berarti perubahan DNS akan menyebar lebih cepat.
Edge Network	Infrastruktur jaringan yang ditempatkan sedekat mungkin dengan pengguna akhir secara geografis, bukan di pusat data tunggal. Cloudflare mengoperasikan ratusan titik kehadiran (Point of Presence/PoP) di seluruh dunia, menjadikannya jaringan edge global yang melayani konten dengan latensi sangat rendah.
HTTP Status Code	Kode tiga digit yang dikembalikan oleh server sebagai bagian dari respons HTTP untuk menunjukkan hasil dari sebuah permintaan. Contoh: 200 OK (berhasil), 201 Created (berhasil membuat sumber daya baru), 400 Bad Request (permintaan tidak valid), 401 Unauthorized (belum terautentikasi), 403 Forbidden (tidak punya izin), 404 Not Found (tidak ditemukan), 503 Service Unavailable (layanan tidak tersedia).
Hybrid Cloud	Model penggunaan infrastruktur cloud yang menggabungkan dua atau lebih platform cloud dari penyedia yang berbeda. Proyek ini menggunakan model hybrid antara Cloudflare (untuk lapisan frontend dan edge) dan Microsoft Azure (untuk lapisan backend, data, keamanan, dan pemantauan).
Priority Routing	Metode penyeimbangan beban pada Azure Traffic Manager di mana semua lalu lintas secara default diarahkan ke endpoint dengan angka prioritas terendah (prioritas 1). Endpoint dengan prioritas lebih tinggi (angka lebih besar) hanya akan menerima lalu lintas apabila endpoint prioritas lebih rendah terdeteksi tidak tersedia.
Proxy (Same-Origin Proxy)	Server perantara yang menerima permintaan dari klien dan meneruskannya ke server tujuan atas nama klien tersebut. 'Same-Origin' berarti browser melihat semua permintaan seolah-olah ditujukan ke domain yang sama (kelompok11cc.my.id), padahal di balik layar proxy meneruskannya ke domain backend yang berbeda.

CI/CD (Continuous Integration / Continuous Deployment)	Praktik pengembangan perangkat lunak modern di mana setiap perubahan kode yang diunggah ke repositori secara otomatis diuji (CI) dan kemudian didistribusikan (deploy) ke lingkungan produksi (CD) tanpa intervensi manual. Proyek ini menggunakan GitHub Actions sebagai platform CI/CD.
Cold Start	Fenomena yang terjadi pada platform serverless (seperti Azure Functions dengan Consumption Plan) di mana sebuah instance fungsi baru perlu diinisialisasi dari awal karena tidak ada instance yang sedang aktif. Cold start dapat menyebabkan permintaan pertama memerlukan waktu respons yang lebih lama (beberapa detik) dibandingkan permintaan berikutnya.
Consumption Plan	Model penetapan harga untuk Azure Functions di mana biaya dikenakan hanya berdasarkan jumlah eksekusi dan sumber daya yang dikonsumsi selama eksekusi berlangsung. Saat tidak ada permintaan masuk, tidak ada biaya yang dikenakan. Cocok untuk aplikasi dengan beban kerja yang tidak dapat diprediksi.
Deployment	Proses mendistribusikan kode aplikasi yang telah selesai dikembangkan ke lingkungan server yang dapat diakses oleh pengguna akhir (lingkungan produksi). Dalam proyek ini, deployment backend dilakukan secara otomatis melalui pipeline GitHub Actions setiap kali ada kode baru yang di-push ke branch 'main'.
Environment Variable (Variabel Lingkungan)	Variabel yang nilainya diatur pada tingkat sistem operasi atau platform (bukan di dalam kode sumber) dan dapat diakses oleh aplikasi saat berjalan. Digunakan untuk menyimpan informasi konfigurasi yang berbeda antara lingkungan development dan produksi, serta untuk menyembunyikan nilai-nilai rahasia dari kode sumber.
GitHub Actions	Platform otomatisasi dari GitHub yang memungkinkan developer untuk mendefinisikan alur kerja (workflow) otomatis dalam format file YAML. Workflow ini akan berjalan secara otomatis saat sebuah peristiwa tertentu terjadi di repositori, seperti ketika ada kode baru yang di-push. Proyek ini menggunakannya untuk otomatisasi deployment backend ke Azure Functions.
Health Check	Pemeriksaan periodik yang dilakukan secara otomatis oleh sistem (seperti Azure Traffic Manager) untuk memastikan bahwa sebuah endpoint masih aktif dan dapat merespons permintaan. Jika health check gagal beberapa kali berturut-turut, Traffic Manager akan menandai endpoint tersebut sebagai tidak sehat dan mengalihkan lalu lintas ke endpoint cadangan.
IaC (Infrastructure as Code)	Pendekatan pengelolaan infrastruktur IT di mana konfigurasi server, jaringan, dan layanan cloud didefinisikan menggunakan file kode (bukan dikonfigurasi secara manual melalui antarmuka grafis). Keuntungannya adalah infrastruktur dapat direproduksi, diaudit, dan dikontrol versinya layaknya kode perangkat lunak biasa.

Repository (Repositori)	Tempat penyimpanan terpusat untuk semua berkas kode sumber, konfigurasi, dan riwayat perubahan sebuah proyek perangkat lunak. Proyek ini menggunakan repositori GitHub yang dapat diakses secara publik.
Terraform	Alat IaC (Infrastructure as Code) open-source yang dikembangkan oleh HashiCorp. Terraform memungkinkan developer untuk mendefinisikan seluruh infrastruktur cloud (resource Azure, jaringan, kebijakan keamanan, dsb.) dalam format kode deklaratif yang disebut HCL (HashiCorp Configuration Language), sehingga infrastruktur dapat dikelola dan direproduksi secara konsisten.
Workflow (GitHub Actions)	Serangkaian langkah otomatis yang didefinisikan dalam sebuah file YAML di dalam direktori <code>.github/workflows/</code> pada repositori. Setiap langkah (step) dalam workflow dapat menjalankan perintah shell, menggunakan action yang sudah tersedia, atau berinteraksi dengan layanan cloud eksternal.

F — Pengujian & Alat Bantu

API (Application Programming Interface)	Antarmuka yang memungkinkan dua sistem perangkat lunak yang berbeda untuk saling berkomunikasi dan bertukar data. Dalam konteks web, API biasanya diimplementasikan sebagai sekumpulan endpoint HTTP yang dapat dipanggil menggunakan metode GET, POST, PUT, PATCH, atau DELETE.
API Key	Kode unik yang digunakan untuk mengidentifikasi dan mengautentikasi aplikasi atau pengguna yang memanggil sebuah API. API Key berfungsi seperti 'kata sandi' untuk mengakses layanan API tertentu.
Black-Box Testing	Metode pengujian perangkat lunak di mana penguji hanya berfokus pada masukan (input) yang diberikan ke sistem dan keluaran (output) yang dihasilkan, tanpa mengetahui atau memeriksa struktur internal kode program. Penguji berperan layaknya pengguna akhir yang tidak tahu bagaimana sistem bekerja di dalamnya.
cURL	Alat baris perintah (command-line tool) yang digunakan untuk melakukan transfer data menggunakan berbagai protokol jaringan, termasuk HTTP dan HTTPS. Sering digunakan oleh developer untuk menguji API secara langsung dari terminal.
Endpoint	URL spesifik pada sebuah server yang merespons jenis permintaan HTTP tertentu. Setiap endpoint biasanya merepresentasikan sebuah sumber daya atau aksi tertentu. Contoh: POST ke <code>/api/upload</code> adalah endpoint untuk mengunggah berkas.
Form-Data	Tipe pengkodean badan permintaan HTTP yang digunakan untuk mengirimkan data formulir, termasuk berkas biner (seperti gambar atau CSV). Dalam Postman, ini dipilih pada tab Body > form-data dan memungkinkan pengiriman berkas langsung ke server.

HTTP (HyperText Transfer Protocol)	Protokol komunikasi standar yang digunakan di World Wide Web untuk pertukaran data antara klien (browser atau aplikasi) dan server. HTTPS adalah versi aman dari HTTP yang menggunakan enkripsi SSL/TLS untuk melindungi data yang dikirimkan.
JSON (JavaScript Object Notation)	Format pertukaran data ringan yang mudah dibaca manusia dan mudah diproses oleh mesin. JSON menggunakan pasangan kunci-nilai (key-value pairs) dan mendukung tipe data seperti string, angka, boolean, array, dan objek bersarang. Hampir semua komunikasi data pada proyek ini menggunakan format JSON.
Postman	Aplikasi desktop populer yang digunakan oleh developer untuk merancang, menguji, mendokumentasikan, dan membagikan API. Postman menyediakan antarmuka grafis yang memudahkan pembuatan permintaan HTTP dengan berbagai metode, header, body, dan autentikasi tanpa perlu menulis kode.
Request / Response	Dua sisi dari sebuah komunikasi HTTP. 'Request' adalah permintaan yang dikirimkan oleh klien ke server (memuat metode, URL, header, dan body). 'Response' adalah jawaban yang dikembalikan oleh server (memuat status code, header, dan body berisi data atau pesan error).
REST API (RESTful API)	Gaya arsitektur untuk merancang API web yang menggunakan protokol HTTP standar dan prinsip-prinsip seperti statelessness (setiap permintaan bersifat independen), penggunaan metode HTTP yang tepat (GET untuk membaca, POST untuk membuat, PATCH untuk memperbarui, DELETE untuk menghapus), dan representasi sumber daya dalam format JSON.

BAGIAN II — ALUR KERJA SISTEM SECARA LENGKAP

Bagian ini menjelaskan bagaimana setiap komponen sistem bekerja sama untuk melayani pengguna—mulai dari saat pengguna membuka browser hingga data berhasil tersimpan dan dapat divisualisasikan. Setiap alur dijelaskan langkah demi langkah menggunakan bahasa yang mudah dipahami.

Alur 1: Pengguna Mengakses Dashboard (Dari Browser ke CDN)

1	Pengguna mengetikkan alamat 'kelompok11cc.my.id' di browser.
2	Browser mengirimkan permintaan DNS ke resolver terdekat untuk menerjemahkan nama domain tersebut menjadi alamat IP.
3	Cloudflare DNS menjawab dengan alamat IP server edge Cloudflare yang paling dekat secara geografis dengan pengguna.
4	Browser terhubung ke server edge Cloudflare dan menerima berkas HTML, CSS, dan JavaScript dashboard langsung dari CDN dengan latensi sangat rendah (tanpa perlu berkomunikasi dengan server Azure sama sekali).
5	Halaman dashboard tampil di layar pengguna.

Alur 2: Pengguna Melakukan Login

1	Pengguna mengisi email dan kata sandi pada formulir login di dashboard, lalu menekan tombol 'Login'.
2	Browser mengirimkan permintaan POST ke '/api/login' di domain Cloudflare Pages.
3	Cloudflare Pages Function (proxy) menangkap permintaan tersebut, menambahkan Function Key Azure secara rahasia, dan meneruskannya ke Azure Traffic Manager.
4	Traffic Manager memverifikasi bahwa backend utama (Azure Functions) sehat, lalu meneruskan permintaan ke sana.
5	Azure Functions mengambil kunci rahasia (AUTH_TOKEN_SECRET) dari Azure Key Vault menggunakan Managed Identity.
6	Azure Functions mencari data pengguna berdasarkan email di kontainer 'users' pada Cosmos DB, lalu membandingkan hash kata sandi yang dikirimkan dengan hash yang tersimpan menggunakan algoritma PBKDF2.
7	Jika cocok, Azure Functions membuat dan menandatangani token JWT yang memuat informasi ID, nama, dan peran pengguna, kemudian mengirimkannya kembali sebagai respons.
8	Browser menerima token JWT dan menyimpannya untuk digunakan pada permintaan-permintaan berikutnya.

Alur 3: Pengguna Mengunggah & Memproses Data (Main Ingestion Flow)

1	Pengguna memilih berkas CSV/JSON/XLSX pada halaman dashboard dan menekan tombol 'Upload & Proses'.
2	Browser mengirimkan permintaan POST ke '/api/upload' beserta berkas dan token JWT di header Authorization.
3	Cloudflare Pages Function meneruskan permintaan (termasuk isi berkas) ke backend Azure Functions melalui Traffic Manager.
4	Azure Functions memvalidasi token JWT, mengekstrak ID pemilik data, lalu membaca dan mem-parsing berkas (CSV/JSON/XLSX) ke dalam memori.
5	Jika parameter 'clean=true' aktif, mesin pembersih data menjalankan serangkaian operasi: pemangkasan spasi teks, konversi nilai kosong ke null, konversi tipe data numerik dan boolean, penghapusan baris duplikat, dan penyaringan outlier menggunakan metode IQR.
6	Salinan berkas mentah diunggah ke kontainer 'raw-data' di Azure Blob Storage sebagai arsip.
7	Setiap rekaman data yang telah bersih ditulis ke kontainer 'telemetry-data' di Azure Cosmos DB, dilengkapi dengan metadata seperti ID pemilik, waktu pemrosesan, kategori, dan status.
8	Azure Functions mengirimkan laporan hasil pemrosesan (cleaning report) sebagai respons JSON '201 Created' kembali ke browser.

Alur 4: Pemrosesan Otomatis via Blob Trigger (Event-Driven Ingestion)

1	Sebuah berkas baru ditulis langsung ke kontainer 'raw-data' di Azure Blob Storage (misalnya oleh skrip otomatisasi atau layanan eksternal).
2	Event system Azure mendeteksi adanya berkas baru dan secara otomatis memicu fungsi 'process_blob' pada Azure Functions.
3	Fungsi process_blob membaca isi berkas dari Blob Storage, mem-parsingnya, dan menjalankan alur pemrosesan data yang sama dengan alur upload manual.
4	Rekaman yang telah diproses ditulis ke Cosmos DB. Tidak ada interaksi HTTP yang diperlukan dari sisi pengguna.

Alur 5: Sistem Failover Otomatis (High Availability)	
1	Azure Traffic Manager melakukan pemeriksaan kesehatan (health check) ke endpoint '/api/hello' pada Azure Functions setiap 30 detik.
2	Jika Azure Functions gagal merespons health check sebanyak 3 kali berturut-turut (total waktu deteksi ± 90 detik), Traffic Manager menandai endpoint utama sebagai 'Degraded' (tidak sehat).
3	Traffic Manager secara otomatis mengubah konfigurasi rutingnya untuk mengarahkan semua lalu lintas baru ke endpoint prioritas 2, yaitu Azure App Service (backup backend).
4	Pengguna yang mengakses '/api/hello' atau '/api/fallback-status' akan menerima respons '200 OK' dari App Service yang menyatakan sistem sedang dalam mode fallback.
5	Permintaan ke endpoint pemrosesan data (seperti '/api/upload') akan menerima respons '503 Service Unavailable' disertai pesan yang menjelaskan bahwa fitur tersebut hanya tersedia di backend utama.
6	Setelah Azure Functions pulih dan berhasil merespons health check, Traffic Manager secara otomatis memulihkan ruting kembali ke backend utama.

Dokumen ini diterbitkan oleh Kelompok 11 sebagai bagian dari Laporan Teknis Akhir mata kuliah Cloud Computing. Seluruh penjelasan disesuaikan dengan implementasi nyata yang berjalan pada platform produksi di Microsoft Azure dan Cloudflare.